Dynamics

Overview

The Dynamics modules (`class_vessel.py` and `calculate_hydrodynamics.py`) implement the mathematical foundation for simulating the motion of marine vehicles through water. The simulator uses Fossen's nonlinear 6-DOF equation of motion:

$$M\dot{\nu} + C(\nu)\nu + D(\nu)\nu + g(\eta) = \tau$$

where,

- $M = M_{RB} + M_A$ is the mass matrix, combining rigid body mass M_{RB} and added mass M_A
- $C(\nu) = C_{RB}(\nu) + C_A(\nu)$ is the Coriolis and centripetal matrix
- $D(\nu)$ is the hydrodynamic damping matrix
- $g(\eta)$ is the gravitational and buoyancy force vector
- τ is the control force vector from thrusters and control surfaces
- $\nu = [u, v, w, p, q, r]^T$ is the velocity vector in the body frame
- $\eta = [x, y, z, \phi, \theta, \psi]^T$ is the position and orientation vector

> **Tip**
>
> The dynamics modules serve as the core of the simulation, determining how forces and moments translate into vessel motion through water.

State Representation

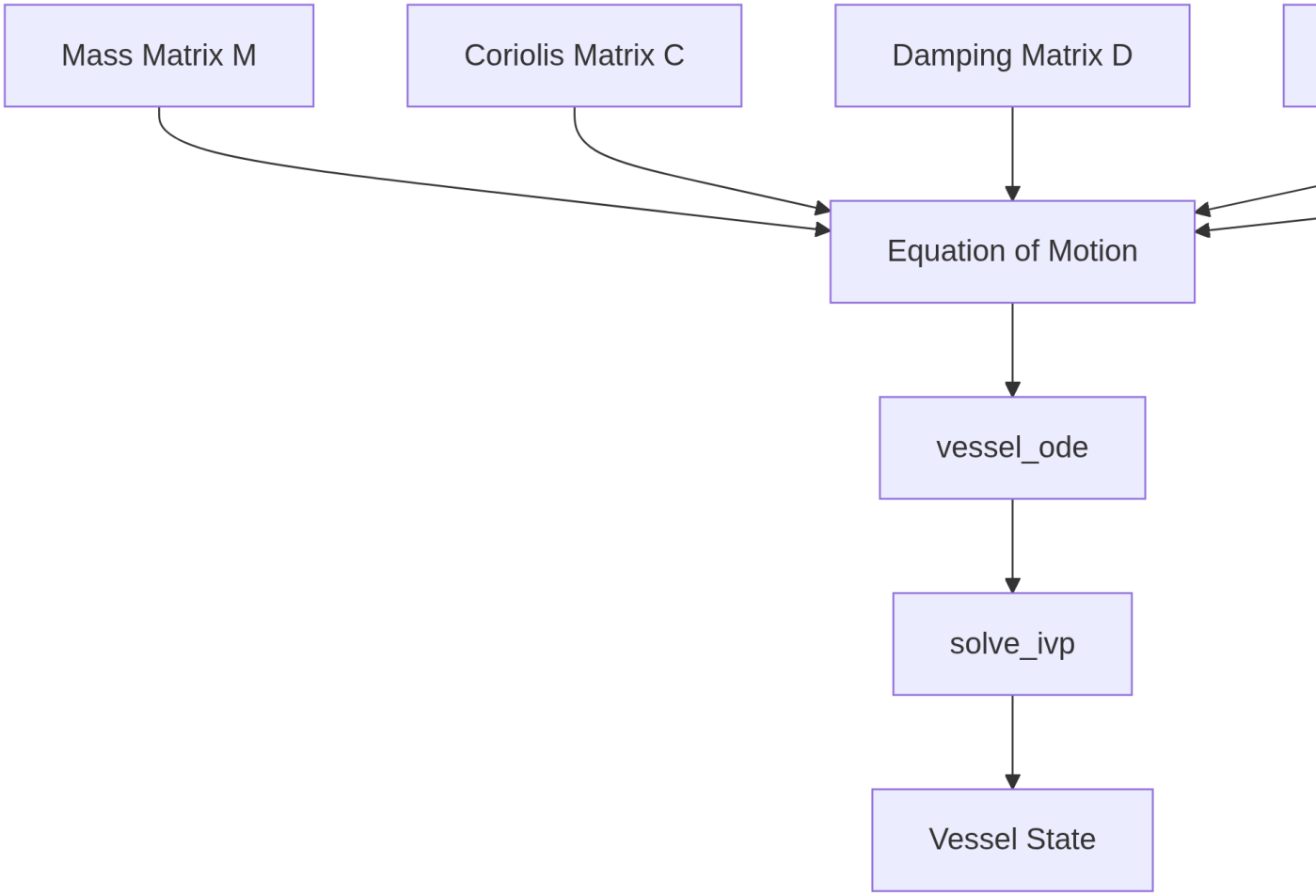
The vessel state is represented as a vector combining velocities, position, orientation, control surface angles, and thruster states:

state $= [\nu^T, \eta_{pos}^T, \eta_{rot}^T, \delta^T, n^T]^T$

Where: - $\nu = [u, v, w, p, q, r]^T$ are the linear and angular velocities in the body frame - $\eta_{pos} = [x, y, z]^T$ is the position in the NED frame - η_{rot} is the orientation, either as Euler angles

$[\phi, \theta, \psi]^T$ or quaternion $[q_0, q_1, q_2, q_3]^T$ - δ is the vector of control surface angles - n is the vector of thruster rotational speeds (RPM)

Components of the Dynamic Equation



Mass Matrix (M)

The mass matrix M combines the rigid-body inertia matrix M_{RB} and the added mass matrix M_A:

$$M = M_{RB} + M_A$$

Rigid-body Mass Matrix (M_{RB})

The rigid-body mass matrix is generated in the `_generate_mass_matrix` method in `calculate_hydrodynamics.py`:

$$M_{RB} = \begin{bmatrix} mI_{3\times3} & -mS(r_G) \\ mS(r_G) & I_G \end{bmatrix}$$

Where: - m is the vessel mass - $I_{3\times3}$ is the 3×3 identity matrix - r_G is the center of gravity vector relative to the origin of the body frame - $S(r_G)$ is the skew-symmetric matrix of r_G (implemented by `Smat()` in `module_kinematics.py`) - I_G is the inertia tensor calculated from the radii of gyration

```
def _generate_mass_matrix(self, CG, mass, gyration):
    # Calculate gyration tensor about vessel frame
    xprdct2 = np.diag(gyration)**2 - Smat(CG)@Smat(CG)

    # Generate the inertia matrix using radii of gyration
    inertia_matrix = xprdct2 * mass

    # Generate the mass matrix
    mass_matrix = np.zeros((6,6))
    mass_matrix[0:3][:, 0:3] = mass * np.eye(3)
    mass_matrix[3:6][:, 3:6] = inertia_matrix
    mass_matrix[0:3][:, 3:6] = -Smat(CG) * mass
    mass_matrix[3:6][:, 0:3] = Smat(CG) * mass

    return mass_matrix
```

Added Mass Matrix (M_A)

The added mass matrix represents the added inertia due to accelerating the surrounding fluid. It's calculated from hydrodynamic data obtained from HydRA (You can also enter custom hydrodynamics added mass coefficients (example Y_vd) in the `hydrodynamics.yml` input file instead of using HydRA, as discussed in the Inputs section).

$$M_A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

Function	Description
`calculate_added_mass_from_hydra(hydra_file)`	Computes the added mass matrix from HydRA data file containing frequency-dependent hydrodynamic coefficients. Extracts zero-frequency added mass for most terms, but computes heave, roll, and pitch terms at their natural frequencies for better accuracy.

```python
def calculate_added_mass_from_hydra(self, hydra_file):
    # Load hydrodynamic data
    with open(hydra_file, 'r') as file:
        mdict = json.load(file)

    # Extract arrays from data
    omg = np.array(mdict['w'])      # Frequency array
    AM = np.array(mdict['AM'])      # Added mass array

    # Extract zero-frequency added mass
    A_zero = AM[1, :, :, 0, 0]

    # Calculate natural frequencies and
    # interpolate added mass at those frequencies
    # ...

    # Return properly transformed added mass matrix
    return A
```

> **Note**
>
> The added mass calculated from HydRA is in ENU frame and so it is converted to NED frame in the `calculate_added_mass_from_hydra` function.

Coriolis and Centripetal Matrix

The Coriolis matrix ($C(\nu)$) accounts for forces arising from motion in a rotating reference frame (Body frame is rotating with respect to NED frame):

$$C(\nu) = C_{RB}(\nu) + C_A(\nu)$$

These matrices are calculated using:

$$C_{RB}(\nu) = \begin{bmatrix} 0_{3\times3} & -S(M_{11}v_1 + M_{12}v_2) \\ -S(M_{11}v_1 + M_{12}v_2) & -S(M_{21}v_1 + M_{22}v_2) \end{bmatrix}$$

$$C_A(\nu) = \begin{bmatrix} 0_{3\times3} & -S(A_{11}v_1 + A_{12}v_2) \\ -S(A_{11}v_1 + A_{12}v_2) & -S(A_{21}v_1 + A_{22}v_2) \end{bmatrix}$$

Where $v_1 = [u, v, w]^T$ and $v_2 = [p, q, r]^T$ are the linear and angular velocity components of ν.

Function	Description
`calculate_coriolis_matrices(vel)`	Computes the rigid body and added mass Coriolis-centripetal matrices based on the current velocity state. Returns a tuple of (C_{RB}, C_A) matrices.

Damping Matrix

The damping matrix ($D(\nu)$) represents hydrodynamic resistance forces and is typically modeled as:

$$D(\nu) = D_{linear} + D_{nonlinear}(\nu)$$

In the implementation, damping is handled through hydrodynamic coefficients (entered in the `hydrodynamics.yml` input file) rather than an explicit matrix:

Function	Description
`hydrodynamic_forces(vel)`	Computes hydrodynamic damping forces by applying coefficients to velocity components. Supports linear, quadratic, and cross-terms.
`cross_flow_drag()`	Estimates sway-yaw damping coefficients for surface vessels using strip theory and Hoerner's cross-flow drag.
`cross_flow_drag_AUV()`	Similar to `cross_flow_drag()` but includes heave-pitch calculations for AUVs.
`hoerner()`	Implements Hoerner's method for computing 2D drag coefficients based on section beam-to-draft ratio.

The coefficients follow a naming convention that indicates which force/moment they affect and which velocity components are involved (you can enter any custom hydrodynamic coefficients in the `hydrodynamics.yml` input file and it will be handled by the function for force calculations):

- Linear damping: `X_u`, `Y_v`, `Z_w`, `K_p`, `M_q`, `N_r`
- Quadratic damping: `X_u_au`, `Y_v_av`, `Z_w_aw`, etc. (the `a` indicates absolute value)
- Cross-coupling: `X_vr`, `Y_ur`, `N_uv`, etc.

```python
def hydrodynamic_forces(self, vel):
    # Extract velocities
    u, v, w, p, q, r = vel

    # Initialize forces vector
    F = np.zeros(6)

    # Map velocity components to values
    vel_map = {'u': u, 'v': v, 'w': w, 'p': p, 'q': q, 'r': r}

    # Process each hydrodynamic coefficient
    for coeff_name, coeff_value in self.hydrodynamics.items():
        if coeff_value == 0: continue

        parts = coeff_name.split('_')
        force_dir = parts[0]
        if force_dir not in self.force_indices: continue

        # Calculate force contribution
        force = coeff_value
        for component in parts[1:]:
            if component.startswith('a') and len(component) > 1:
                # Absolute value term (e.g., u|u|)
                v_char = component[1:]
                force *= abs(vel_map[v_char])
            elif component in vel_map:
                # Regular term
                force *= vel_map[component]

        # Add to force vector
        F[self.force_indices[force_dir]] += force

    return F
```

Restoring Forces

The restoring forces ($g(\eta)$) include gravitational and buoyancy effects:

$$g(\eta) = \begin{bmatrix} (W-B)\sin\theta \\ -(W-B)\cos\theta\sin\phi \\ -(W-B)\cos\theta\cos\phi \\ -(y_G W - y_B B)\cos\theta\cos\phi + (z_G W - z_B B)\cos\theta\sin\phi \\ (z_G W - z_B B)\sin\theta + (x_G W - x_B B)\cos\theta\cos\phi \\ -(x_G W - x_B B)\cos\theta\sin\phi - (y_G W - y_B B)\sin\theta \end{bmatrix}$$

Where:

- $W = mg$ is the weight
- $B = \rho g \nabla$ is the buoyancy force (with ∇ being the displaced volume)
- $r_G = [x_G, y_G, z_G]^T$ is the center of gravity mentioned in the `geometry.yml` input file
- $r_B = [x_B, y_B, z_B]^T$ is the center of buoyancy mentioned in the `geometry.yml` input file

Function	Description
`gravitational_forces(phi, theta)`	Computes the gravitational and buoyancy forces and moments as a function of roll and pitch angles.

Control Forces

The control forces vector τ represents the combined effect of control surfaces and thrusters.#### Control Surface Forces

Function	Description
`control_forces(delta)`	Calculates forces and moments generated by control surfaces (rudders, fins, etc.) based on their deflection angles δ. Supports both hydrodynamic coefficient-based and aerofoil-based calculation methods.

For hydrodynamic coefficient-based calculation: - Forces are calculated as $\tau_i = C_{i\delta}\delta$, where $C_{i\delta}$ is the control coefficient mentioned in the `control_surface_hydrodynamics` in the `control_surfaces.yml` input file

For aerofoil-based calculation:

- Local velocity at the control surface is calculated considering vessel motion (automatically done)
- Angle of attack is determined from flow direction and surface deflection
- Lift and drag forces are calculated using NACA airfoil data (path to the NACA file is mentioned in the `control_surfaces.yml` input file)
- Forces are transformed to the body frame and a generalized force vector is returned

Thruster Forces

Function	Description
`thruster_forces(n_prop)`	Calculates forces and moments from propellers/thrusters based on their rotational speeds n in RPM. Models thrust using propeller theory with advance ratio and thrust coefficients.

The thrust calculation follows the form:

$$X_{prop} = K_T \rho D^4 |n| n$$

Where:

- K_T is the thrust coefficient (approximated as $K_{T,J=0}(1 - J)$ with J being the advance ratio)
- ρ is the water density
- D is the propeller diameter
- n is the propeller speed in revolutions per second

Vessel ODE and Simulation

The vessel dynamics are implemented as an ordinary differential equation (ODE) in the `vessel_ode(t, state)` method, which computes the state derivatives:

```
def vessel_ode(self, t, state):
    # Extract state components
    vel = state[0:6]  # [u, v, w, p, q, r]
    pos = state[6:9]  # [x, y, z]
    angles = state[9:attitude_end]  # [phi, theta, psi] or quaternion

    # Calculate forces and moments
```

```
    F_hyd = self.hydrodynamic_forces(vel)
    F_control = self.control_forces(state[control_start:thruster_start])
    F_thrust = self.thruster_forces(state[thruster_start:])
    F_g = self.gravitational_forces(angles[0], angles[1])

    if self.coriolis_flag:
        C_RB, C_A = self.calculate_coriolis_matrices(vel)
        F_C = (C_RB + C_A) @ vel
    else:
        F_C = np.zeros(6)

    # Total force
    F = F_hyd + F_control + F_thrust - F_g - F_C

    # Calculate velocity derivatives
    M = self.mass_matrix + self.added_mass_matrix  # Total mass matrix
    state_dot[0:6] = np.linalg.inv(M) @ F

    # Calculate position and attitude derivatives
    # ...

    return state_dot
```

Function	Description
`step()`	Advances the simulation by one time step by solving the ODE using SciPy's `solve_ivp`.
`simulate()`	Runs the full simulation by repeatedly calling `step()` until the final time.
`reset()`	Resets the vessel to its initial state.

Helper Methods and Utilities

Dimensionalization

Hydrodynamic coefficients can be provided in non-dimensional form and converted to dimensional form:

Function	Description
`_dimensionalize_coefficients(rho, L, U)`	Converts non-dimensional hydrodynamic coefficients to dimensional form based on density ρ, length L, and characteristic velocity U (mentioned in the `simulation_input.yml` file)

Hydrodynamic Coefficient Calculation

Function	Description
`calculate_added_mass_from_hydra(hydra_files)`	Calculates the added mass matrix from HydRA data files.
`cross_flow_drag()`	Estimates sway-yaw hydrodynamic coefficients using strip theory.
`cross_flow_drag_AUV()`	Estimates both sway-yaw and heave-pitch coefficients for underwater vehicles.
`hoerner()`	Implements Hoerner's method for 2D section drag coefficient calculation.

> **Note**
>
> Understanding these dynamics components is essential for correctly configuring vessel parameters and interpreting simulation results. The implementation allows for flexible modeling of different vessel types by adjusting the parameters and coefficients.